



RIFCO

MVP Execution Roadmap and Development Packet

Phase based build guide, checklist, and milestone tracker for the MVP build

Build model	Solo founder build, with you as the only developer for the MVP
Locked stack	React Native, Firebase, native modules for location and tracking, Next.js on Vercel for web and ops
Launch wedge	Oyster shucker only at launch, later expand into broader experience categories

1. How to Use This Roadmap

This document is the execution version of the RIFCO blueprint. It is built so you can move through the MVP in order, assign yourself a phase, and know exactly what must be completed before the next stage starts.

- Use each phase as a lock gate. Do not begin the next phase until the current one has a clear output.
- Treat every checklist as a completion tracker. Check items only when the work is truly done and tested.
- Use the weekly targets as pacing guidance, not as a reason to rush unfinished foundations.
- Because you are the only developer, the roadmap keeps scope tight and protects you from bouncing between unrelated build tasks.

2. Locked Product Direction

RIFCO launches as a simple consumer app for requesting an oyster shucker. The product is intentionally narrow at launch so the first version can collect clean operational data, train pricing logic, and prove the flow before the app opens into broader experience categories.

Scope rule for MVP

No expansion into bartender, chef, magician, musician, or other categories until the oyster shucker flow, pricing system, dispatch logic, tracking, and data capture are working reliably in production.

3. Locked Tech Stack

- Mobile app: React Native with TypeScript
- Native modules: Swift on iOS and Kotlin on Android for tracking, maps, background location, and other sensitive live workflows
- Backend: Firebase Auth, Firestore, Cloud Functions, Cloud Storage, Firebase Cloud Messaging, Firebase Analytics, and likely Realtime Database for provider presence
- Web and internal tools: Next.js on Vercel
- Maps and routing: Google Maps Platform
- Payments: Stripe

4. Phase Overview

Phase	Focus	Target	Gate	Main outcome
1	Core setup and planning	Week 1 to 2	Gate A	All accounts, environments, scope, and schemas are locked
2	Product flow and screens	Week 2 to 3	Gate B	User journey and screen map are final
3	Project foundation build	Week 3 to 4	Gate C	Apps, repos, configs, auth, and navigation are running
4	Customer app core flow	Week 4 to 7	Gate D	Customer can request, price, book, and pay
5	Provider app core flow	Week 5 to 8	Gate E	Provider can onboard, go online, accept, and complete
6	Realtime tracking and maps	Week 6 to 9	Gate F	Tracking flow works with live state updates
7	Algorithm and data engine	Week 7 to 10	Gate G	Pricing and event logic are active and logging
8	Ops dashboard and controls	Week 8 to 11	Gate H	You can manage providers, requests, and pricing
9	QA, polish, and launch prep	Week 10 to 14	Gate I	Core bugs are cleared and launch systems are ready
10	Soft launch and calibration	Week 14 to 17	Gate J	Production launch begins with close monitoring

Phase 1. Core Setup and Planning

Objective

Create the technical foundation, project structure, and product rules before a single major screen is built.

Checklist

Done	Task	Priority	Depends on
☐	Create the Firebase project, enable billing, and confirm the correct account ownership model	Critical	None
☐	Enable Firebase Auth, Firestore, Cloud Storage, Cloud Functions, Cloud Messaging, and Analytics	Critical	Firebase project
☐	Decide whether to also enable Realtime Database for provider presence and connection status	High	Firebase project
☐	Create the Google Cloud project and connect the Google Maps Platform APIs you need	Critical	Firebase and Google project
☐	Enable Maps SDK, Places, Geocoding, Directions, and any related quota alerts	High	Google Cloud project
☐	Create the Stripe account, connect business settings, and outline payment flow assumptions	Critical	None
☐	Create the React Native app shell and set up iOS and Android builds locally	Critical	None
☐	Create the Vercel project and the Next.js internal ops dashboard shell	Medium	None
☐	Write the locked MVP scope so the build remains oyster shucker only	Critical	None
☐	Write the request lifecycle states, provider states, cancellation states, and payment states	Critical	Locked scope
☐	Write the first Firestore collection map and event logging taxonomy	Critical	Locked scope
☐	Write the algorithm input list for pricing, demand, distance, and supply logic	High	Locked scope

Completion Gate

Gate A

You should have all accounts live, billing active, quotas visible, repositories initialized, and the first product and data definitions written down.

Phase 2. Product Flow and Screen Mapping

Objective

Design the product flow from start to finish so development follows one clear path instead of random feature building.

Checklist

Done	Task	Priority	Depends on
☐	Write the complete customer journey from app open through request completion	Critical	Gate A
☐	Write the complete provider journey from signup through job completion	Critical	Gate A
☐	List every screen needed for MVP in the customer app	Critical	Gate A
☐	List every screen needed for MVP in the provider app	Critical	Gate A
☐	Define onboarding questions, profile fields, and permissions flow	High	Gate A
☐	Define the request form fields, pricing view, confirmation screen, tracking screen, and completion flow	Critical	Gate A
☐	Define provider profile, availability screen, incoming request screen, navigation state, and earnings history screen	High	Gate A
☐	Define all status badges, labels, and user facing language for live jobs	Medium	Gate A
☐	Create a screen order map so you always know what to build next	Critical	Gate A

Completion Gate

Gate B

You should know every screen, every major state, and the exact order in which the product is built.

Phase 3. Project Foundation Build

Objective

Turn the plan into a stable codebase with working navigation, environments, authentication, and baseline services.

Checklist

Done	Task	Priority	Depends on
☐	Set up environment variable handling for Firebase, Stripe, and Google Maps	Critical	Gate B
☐	Configure React Native navigation structure for customer and provider areas	Critical	Gate B
☐	Connect Firebase SDKs and confirm auth, database, storage, messaging, and analytics integration	Critical	Gate A
☐	Create the app theme, type scale, color tokens, spacing system, and shared UI components	High	Gate B
☐	Build reusable inputs, buttons, cards, modals, loading states, and error states	High	Gate B
☐	Set up auth flows for email, phone, or social sign in based on final decision	Critical	Gate A
☐	Create base Firestore rules and storage rules for development	Critical	Gate A
☐	Create the first cloud function scaffolding and deployment flow	High	Gate A
☐	Set up crash reporting and logging for early testing	High	Gate A

Completion Gate

Gate C

The codebase should feel like a real product foundation, not a loose experiment.

Phase 4. Customer App Core Flow

Objective

Build the customer side first so the core commercial booking flow exists early.

Checklist

Done	Task	Priority	Depends on
☐	Build customer onboarding and profile setup	Critical	Gate C
☐	Build location permissions and location capture flow	Critical	Gate C
☐	Build the request creation screen for oyster shucker service	Critical	Gate B
☐	Build date and time selection logic for immediate and scheduled requests	High	Gate B
☐	Build pricing quote display with distance, time, and demand inputs	Critical	Gate A
☐	Build booking confirmation and payment method flow	Critical	Gate A
☐	Integrate Stripe payment intent flow	Critical	Gate A
☐	Build request status screen with state changes from created through assigned	Critical	Gate C
☐	Build booking history and request detail screens	Medium	Gate C
☐	Log all customer events into analytics and event collections	Critical	Gate A

Completion Gate

Gate D

A customer should be able to sign up, request service, see a quote, pay, and watch the request progress.

Phase 5. Provider App Core Flow

Objective

Build the provider side so the supply side can be onboarded and dispatched.

Checklist

Done	Task	Priority	Depends on
☐	Build provider signup and profile creation	Critical	Gate C
☐	Build provider onboarding fields, service area, documents, and approval status	Critical	Gate B
☐	Build provider home screen with online and offline state	Critical	Gate C
☐	Build provider availability controls and active shift state	High	Gate C
☐	Build incoming request screen with accept and decline actions	Critical	Gate D
☐	Build provider job detail view with location, event details, and customer info	Critical	Gate D
☐	Build job progress states for accepted, en route, arrived, started, completed, and canceled	Critical	Gate B
☐	Build provider earnings summary and past jobs screen	Medium	Gate D
☐	Log provider response time, acceptance, decline, completion, and cancellation events	Critical	Gate A

Completion Gate

Gate E

A provider should be able to onboard, go online, receive work, complete jobs, and feed data back into the system.

Phase 6. Realtime Tracking and Maps

Objective

Add the live operational layer that makes the product feel like an on demand service app.

Checklist

Done	Task	Priority	Depends on
☐	Integrate map rendering and geocoding into customer and provider views	Critical	Gate C
☐	Build live request tracking state updates from Firestore listeners	Critical	Gate D
☐	Build provider location update pipeline	Critical	Gate E
☐	Create native modules for background location behavior where needed	Critical	Gate C
☐	Create native modules or native support for map heavy workflows if React Native performance is not enough	High	Gate C
☐	Build arrival status updates and ETA presentation	High	Gate E
☐	Test background behavior, battery impact, and permission edge cases on iOS and Android	Critical	Gate C
☐	Create connection recovery and stale tracking fallback logic	High	Gate E

Completion Gate

Gate F

The product should now feel live, trackable, and operationally believable.

Phase 7. Algorithm and Data Engine

Objective

Turn the product into a data collecting machine and launch the first pricing and dispatch intelligence layer. Checklist

Done	Task	Priority	Depends on
☐	Create collections or logs for pricing decisions, zone snapshots, supply snapshots, and demand snapshots	Critical	Gate A
☐	Write the first quote formula using base fee, distance, time, demand, and availability	Critical	Gate A
☐	Write the first surge logic thresholds by zone or geo area	Critical	Gate A
☐	Write the first provider ranking logic using availability, distance, reliability, and response behavior	Critical	Gate E
☐	Log every request state transition as an event	Critical	Gate D
☐	Log every provider state transition as an event	Critical	Gate E
☐	Run scheduled functions for snapshots, metrics rollups, and recalculation jobs	High	Gate A
☐	Create admin visible records for quote inputs and outputs so you can audit pricing behavior	High	Gate H
☐	Create first dashboards or summaries for demand, completion, cancellation, and pricing behavior	High	Gate H

Completion Gate

Gate G

The app is now collecting the data needed to improve pricing, dispatch, and future algorithm accuracy.

Phase 8. Ops Dashboard and Controls

Objective

Build the internal command center that lets you control the MVP without touching production data manually.

Checklist

Done	Task	Priority	Depends on
☐	Build provider approval and rejection controls	Critical	Gate E
☐	Build request monitoring table with live status	Critical	Gate F
☐	Build provider monitoring table with online status and recent activity	High	Gate F
☐	Build pricing control panel for base fee, distance logic, and surge thresholds	Critical	Gate G
☐	Build support view for customer and provider records	Medium	Gate D
☐	Build metrics view for requests, assignments, completions, cancellations, and revenue signals	High	Gate G
☐	Build audit views for quotes and algorithm output	High	Gate G

Completion Gate

Gate H

You can operate the business and monitor the system without guessing.

Phase 9. QA, Polish, and Launch Prep

Objective

Stabilize the MVP, reduce launch risk, and ensure the narrow oyster shucker launch is operationally tight.

Checklist

Done	Task	Priority	Depends on
☐	Run full end to end tests for customer flow	Critical	Gate H
☐	Run full end to end tests for provider flow	Critical	Gate H
☐	Run live booking simulations with multiple devices	Critical	Gate F
☐	Test quote accuracy against expected outcomes in different zones and times	Critical	Gate G
☐	Test payments, cancellations, and completion states	Critical	Gate D
☐	Review all analytics events to ensure nothing important is missing	High	Gate G
☐	Polish copy, UI gaps, empty states, error states, and edge cases	High	Gate F
☐	Prepare App Store and Google Play assets, copy, policies, and screenshots	Critical	Gate C
☐	Prepare launch checklist, provider onboarding checklist, and support checklist	High	Gate H

Completion Gate

Gate I

The MVP should now be shippable, narrow, and testable with confidence.

Phase 10. Soft Launch and Calibration

Objective

Launch in a controlled way, watch the numbers closely, and tune the engine with real use.

Checklist

Done	Task	Priority	Depends on
☐	Launch with a tightly controlled city and provider group	Critical	Gate I
☐	Monitor demand by zone, provider supply, acceptance speed, cancellations, and completion rate daily	Critical	Gate G
☐	Monitor quote acceptance and user drop off around pricing screens	Critical	Gate G
☐	Tune surge thresholds, distance logic, and quote messaging based on real behavior	Critical	Gate G
☐	Document every operational issue and convert each one into a product or logic fix	High	Gate I
☐	Do not expand categories until the oyster shucker model is stable	Critical	Gate I

Completion Gate

Gate J

The goal is not broad scale yet. The goal is operational truth, clean data, and a stable engine.

5. Suggested Solo Founder Build Order

When you are building alone, sequence matters more than speed. Use this order to keep yourself from context switching too much.

1. Finish every account, billing, API, and environment setup first.
2. Lock the screen map and the user journey before major UI work starts.
3. Build the app foundation and shared components next.
4. Build the customer flow before the provider flow.
5. Build provider flow before live tracking complexity.
6. Integrate maps and live tracking only after the request and provider flows already exist.
7. Turn on the data and pricing engine before launch, not after launch.
8. Build ops controls before public release so you can manage the product cleanly.
9. Leave polish, app store packaging, and launch prep for the final stretch.

6. Weekly Execution Rhythm for One Developer

- Start each week by choosing one active phase and one backup task only.
- Define the week around one milestone, such as customer booking flow complete or provider online state complete.
- Keep one running issue list and one running next build list.
- At the end of each week, write what was finished, what broke, what is blocked, and what the next build target is.
- Do not start a new feature just because it feels exciting. Move only when the current gate is cleared.

7. Whiteboard Mind Map Structure

Use the structure below when you turn this into a visual map on your whiteboard.

- Center node: RIFCO MVP
- Branch 1: Product scope, oyster shucker only, later category expansion
- Branch 2: Tech stack, React Native, Firebase, native modules, Vercel, Google Maps, Stripe
- Branch 3: Customer flow, onboarding, request, pricing, payment, tracking, completion
- Branch 4: Provider flow, signup, approval, online state, incoming request, navigation, completion
- Branch 5: Realtime layer, tracking, maps, status changes, background location
- Branch 6: Algorithm layer, base fee, distance fee, time logic, surge logic, provider ranking
- Branch 7: Data layer, events, snapshots, quotes, completions, cancellations
- Branch 8: Ops dashboard, provider approval, live requests, pricing controls, audit logs
- Branch 9: Launch path, QA, soft launch, calibration, expansion later

8. Final Reminder

Protect the wedge

The MVP wins by being narrow and reliable. Oyster shucker only is not a limitation. It is how the engine gets trained correctly. The broader experience product comes later, after the workflow, tracking, pricing, and data systems prove themselves.